

## FICHE DE TD 3/3 • DIFFICILE

## Informatique • 1re C D E

**Programmation en langage C**

## Module 2 : Algorithmique et programmation

Cette fiche d'évaluation synthétique mobilise l'ensemble des notions du module (structure, bibliothèques, E/S formatées, adressage, boucles, tableaux) pour résoudre des problèmes algorithmiques complets.

**EXERCICE 1 Analyse numérique et boucles conditionnelles**

On souhaite approcher la racine carrée d'un réel strictement positif  $A$  par la suite de Héron (ou de Babylone) définie par :  $U_0 = A$  et  $U_{n+1} = \frac{1}{2} \left( U_n + \frac{A}{U_n} \right)$ . L'algorithme s'arrête lorsque la différence absolue entre deux termes successifs devient inférieure à une précision  $\varepsilon = 10^{-6}$ .

1. Écrire la structure complète d'un programme C (directives `#include` nécessaires, fonction `main`) qui déclare les variables adaptées. Justifier le choix du type pour  $\varepsilon$  et  $A$ .
2. Implémenter la saisie sécurisée de  $A$  (rejeter les valeurs  $\leq 0$ ) en utilisant une boucle `do...while`. Afficher le message d'erreur approprié.
3. Coder le calcul itératif avec une boucle `while`. Veiller à utiliser l'opérateur d'adressage `&` et le spécificateur de format correct dans `scanf` pour la saisie, et `%f` (ou `%lf`) dans `printf` pour afficher l'approximation finale  $U_n$  et le nombre d'itérations  $n$ .
4. Analyse critique : Que se passe-t-il si l'utilisateur saisit une chaîne de caractères au lieu d'un nombre ? Proposer une modification du code (sans `scanf_s`) pour vider le tampon d'entrée (`stdin`) en cas d'échec de `scanf`.

**EXERCICE 2 Manipulation de tableaux, adressage et bibliothèques**

Soit un tableau statique  $T$  de  $N$  entiers ( $N \leq 100$ ). On veut partitionner  $T$  en deux nouveaux tableaux `Pairs` et `Impairs` conservant l'ordre d'apparition, puis calculer l'écart-type  $\sigma$  des valeurs paires.

1. Déclarer les trois tableaux et les compteurs nécessaires. Écrire une boucle `for` de remplissage de  $T$  avec des valeurs aléatoires entre 1 et 1000 en utilisant `rand()` (bibliothèque `stdlib.h`) et `srand()` initialisé avec `time(NULL)` (bibliothèque `time.h` à inclure).
2. Écrire l'algorithme de partitionnement en une seule passe sur  $T$ . Utiliser l'arithmétique des pointeurs (notation `*(T+i)`) pour l'accès aux éléments plutôt que l'indexation `T[i]`.
3. Implémenter une fonction `float ecartType(int *tab, int taille)` recevant

l'adresse du tableau `Pairs` (mode d'adressage par pointeur). La fonction calcule la moyenne  $\mu$  puis  $\sigma = \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$  en utilisant `sqrt` et `pow` de `math.h`.

4. Question ouverte : Lors de la compilation avec `gcc`, l'édition de liens échoue pour `sqrt`. Quelle option de compilation ajouter ? Expliquer le rôle de `conio.h` et de `getch()` en fin de `main` dans un IDE Windows classique (ex : Code ::Blocks).

### EXERCICE 3 Tri, recherche et complexité – Synthèse

On considère un tableau `Notes[30]` de réels (notes sur 20) saisis par l'utilisateur. Le programme doit classer les notes par ordre décroissant, afficher le classement (rang, note), et permettre la recherche d'une note cible.

1. Proposer un tri à bulles optimisé (drapeau d'échange) imbriquant deux boucles `for`. Justifier l'utilisation du passage par adresse (pointeurs) dans une fonction d'échange `void echange(float *a, float *b)` appelée depuis le tri.
2. Après tri, écrire une recherche dichotomique itérative (`while`) pour trouver une note `Cible`. Retourner l'indice (rang) ou `-1`. Attention : le tableau est décroissant, adapter les conditions de branchement.
3. Analyse de robustesse : Si deux élèves ont la même note, la recherche dichotomique standard ne garantit pas de trouver la première occurrence. Modifier l'algorithme pour retourner le premier rang (meilleur classement) correspondant à `Cible`.
4. Débogage IDE : Lors du test, le programme affiche des valeurs aberrantes (ex : `-nan(ind)`) pour certaines notes. Citer trois causes probables liées à l'initialisation, aux bornes de boucles ou aux spécificateurs `printf/scanf`, et décrire la procédure pas-à-pas (points d'arrêt, espionnage variables) dans l'IDE pour localiser la ligne fautive.

### EXERCICE 4 Simulation de gestion de stock – Projet intégré

On modélise un stock de 50 produits. Chaque produit a un `Code` (entier 5 chiffres), un `Prix` (float) et un `Quantite` (entier). Les données sont stockées dans trois tableaux parallèles `Codes[50]`, `Prix[50]`, `Qtes[50]`. Un compteur `NbProd` suit le nombre réel de produits.

1. Écrire une fonction `int Rechercher(int code, int *tab, int n)` renvoyant l'indice d'un code ou `-1`. L'utiliser dans une procédure `Ajouter()` qui vérifie l'unicité du code (erreur si doublon) et l'espace disponible avant insertion en fin de tableaux.
2. Coder `Vendre(int code, int qte)` : recherche le produit, vérifie le stock suffisant ( $\geq qte$ ), décrémente `Qtes`, calcule et affiche le montant total (`Prix * qte`) avec deux décimales. Gérer les codes inexistants et ruptures de stock.

3. Critique de conception : Les tableaux parallèles posent des problèmes de cohérence lors des tris ou suppressions. Proposer une structure `struct Produit` regroupent les trois champs. Réécrire la déclaration du tableau principal et l'appel de `Rechercher` adapté (passage de l'adresse du champ `Code` via `&tab[i].Code` ou passage du tableau de structures).
4. Extension fichier (optionnel) : Donner les instructions `fopen`, `fwrite`, `fclose` pour sauvegarder binairement le tableau de structures dans `stock.dat`. Préciser le mode d'ouverture et l'argument `sizeof`.